LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2



Disusun Oleh :

Muhamad Ramadhani (25782045)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. **Sensor DHT11/DHT22:** Hubungkan pin **VCC** ke **3V3**, pin **GND** ke **GND**, dan pin **Data** ke **D4 (GPIO 2)**.

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, buat berkas sketsa baru (*File > New Sketch*).
2. Tuliskan kode program berikut:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_NIM01";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/nim_anda/telemetri";

// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000; // Interval pengiriman 5 detik

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
    } else {
      Serial.print(" Gagal, kode status rc=");
      Serial.print(client.state());
      Serial.println(". Mencoba kembali dalam 5 detik...");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
}

void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Memelihara sesi TCP dan keep-alive ping

  unsigned long currentMillis = millis();
  if (currentMillis - previousPublishTime >= publishInterval) {
    previousPublishTime = currentMillis;

    float t = dht.readTemperature();
    float h = dht.readHumidity();
```

```
    // Proteksi pembacaan sensor DHT
    if (isnan(t) || isnan(h)) {
      Serial.println("Peringatan: Gagal membaca data DHT!");
      return;
    }

    // Alokasi dokumen JSON statis
    StaticJsonDocument<256> doc;
    doc["node_id"] = clientId;
    doc["suhu"] = serialized(String(t, 2));       // Format desimal 2 angka di belakang koma
    doc["kelembapan"] = serialized(String(h, 2));

    // Serialisasi JSON ke dalam array buffer karakter
    char jsonBuffer[256];
    serializeJson(doc, jsonBuffer);

    // Publikasi paket data ke Broker
    bool isSuccess = client.publish(topicPubTelemetry, jsonBuffer);
    if (isSuccess) {
      Serial.print("Telemetri Terkirim [");
      Serial.print(topicPubTelemetry);
      Serial.print("]: ");
      Serial.println(jsonBuffer);
    } else {
      Serial.println("Peringatan: Gagal memublikasikan paket!");
    }
  }
}
```

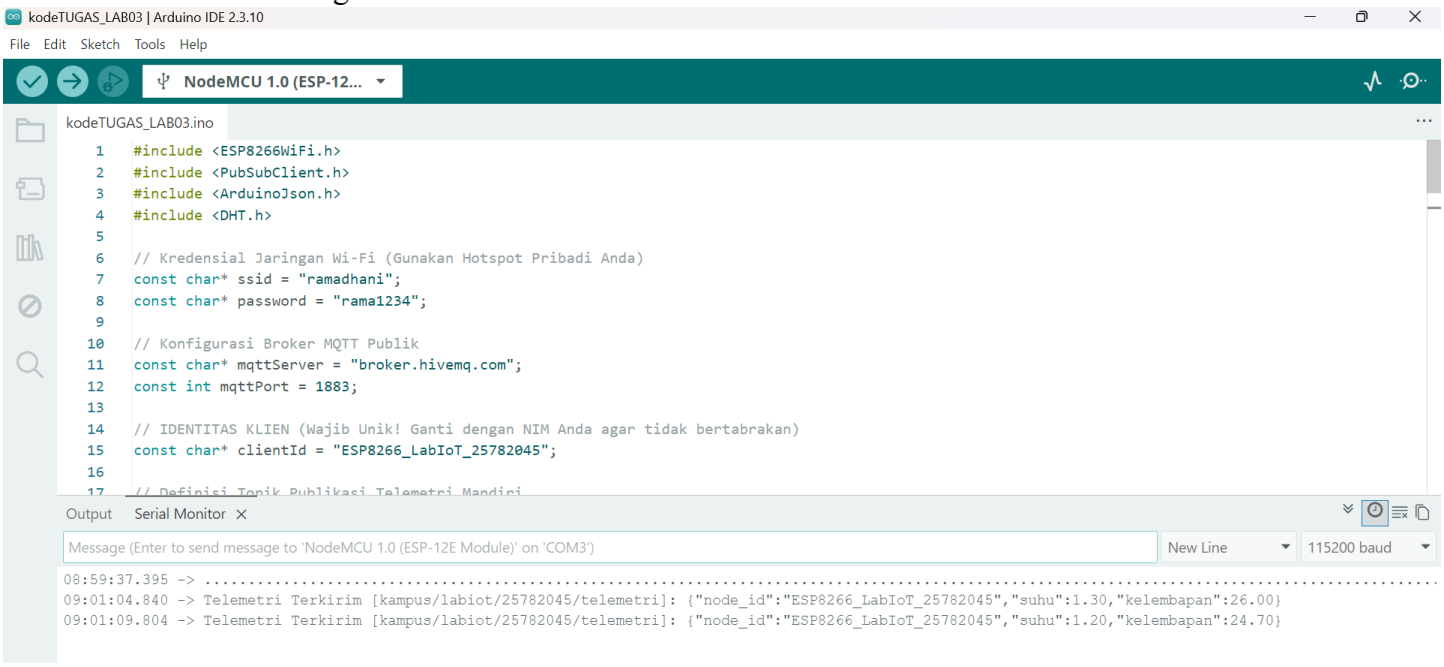
3. Ganti NAMA_WIFI_ANDA, PASSWORD_WIFI_ANDA, clientId (gunakan NIM Anda), dan topik kampus/labiot/nim_anda/telemetri dengan NIM pribadi Anda.
4. Unggah (*upload*) program ke NodeMCU ESP8266, buka Serial Monitor pada *baud rate* **115200**.
5. Amati log: ESP8266 akan terhubung ke Wi-Fi, melakukan jabat tangan dengan broker broker.hivemq.com, dan memublikasikan paket JSON setiap 5 detik.

Penjelasan Singkat Kode:

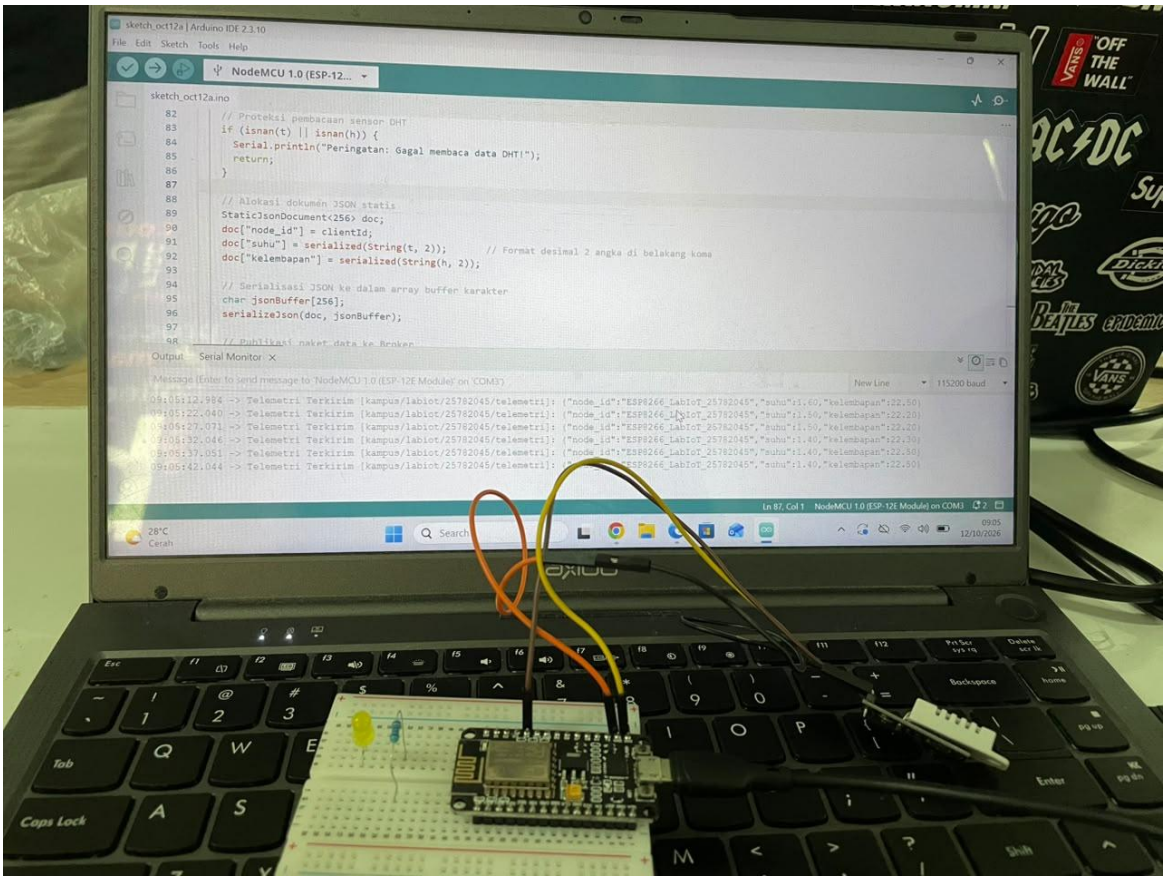
- WiFiClient espClient; : Membuka antarmuka soket jaringan TCP level dasar pada cip ESP8266.
- PubSubClient client(espClient); : Menginisialisasi objek protokol MQTT yang beroperasi di atas soket jaringan espClient.
- client.setServer(mqttServer, mqttPort); : Menetapkan alamat host broker dan port layanan (1883 merupakan port standar MQTT teks terbuka / tanpa TLS).
- client.loop(); : **Fungsi wajib.** Rutinitas ini bertugas memproses paket TCP masuk, menjaga interval *keep-alive ping* dengan server agar koneksi tidak diputus paksa (*timeout*), serta mengelola antrean pesan.
- StaticJsonDocument<256> doc; : Mengalokasikan blok memori tetap di SRAM (sebesar 256 byte) untuk menampung pohon objek JSON tanpa memicu fragmentasi memori *heap*.

- serializeJson(doc, jsonBuffer); : Mengubah pohon objek JSON menjadi representasi string karakter padat yang siap ditransmisikan.

Hasil dokumentasi dari tugasbisa



Bisa dlihat program berhasi mengirim telementri suhu dan kelembapan



6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ωyang menuju ke **GND**.

Langkah Kerja Pemrograman:

1. Buat sketsa baru di Arduino IDE, lalu masukkan kode berikut:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_NIM01_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/nim_anda/perintah";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
}
```

```
// Callback otomatis: Berjalan saat mendeteksi paket masuk pada topik terdaftar
void onMessageReceived(char* topic, byte* payload, unsigned int length) {
  Serial.println("=========================================");
  Serial.print("Pesan Baru Masuk pada Topik: ");
  Serial.println(topic);

  // Ubah deretan byte payload menjadi string aman dengan null-terminator
  char payloadStr[length + 1];
  memcpy(payloadStr, payload, length);
  payloadStr[length] = '\0';

  Serial.print("Payload Diterima: ");
  Serial.println(payloadStr);

  // Deserialisasi (Parsing) JSON
  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, payloadStr);

  if (error) {
    Serial.print("Gagal parsing JSON: ");
    Serial.println(error.f_str());
    return;
  }

  // Evaluasi Parameter 'led' (Format yang diharapkan: {"led": 1} atau {"led": 0})
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> Aksi: Status LED diubah menjadi %s\n", stateLED == 1 ? "MENYALA (ON)" : "PADAM
(OFF)");
  }
  Serial.println("=========================================");
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Mencoba koneksi MQTT...");
    if (client.connect(clientId)) {
      Serial.println(" Terhubung!");
      // Segera daftarkan subskripsi topik perintah sesaat setelah koneksi pulih
      client.subscribe(topicSubCommand);
      Serial.print("Subskripsi Topik: ");
      Serial.println(topicSubCommand);
    } else {
      Serial.printf("Gagal (rc=%d). Ulangi dalam 5 detik...\n", client.state());
```

```
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  // Status awal LED padam
  digitalWrite(ledPin, LOW);

  connectToWiFi();

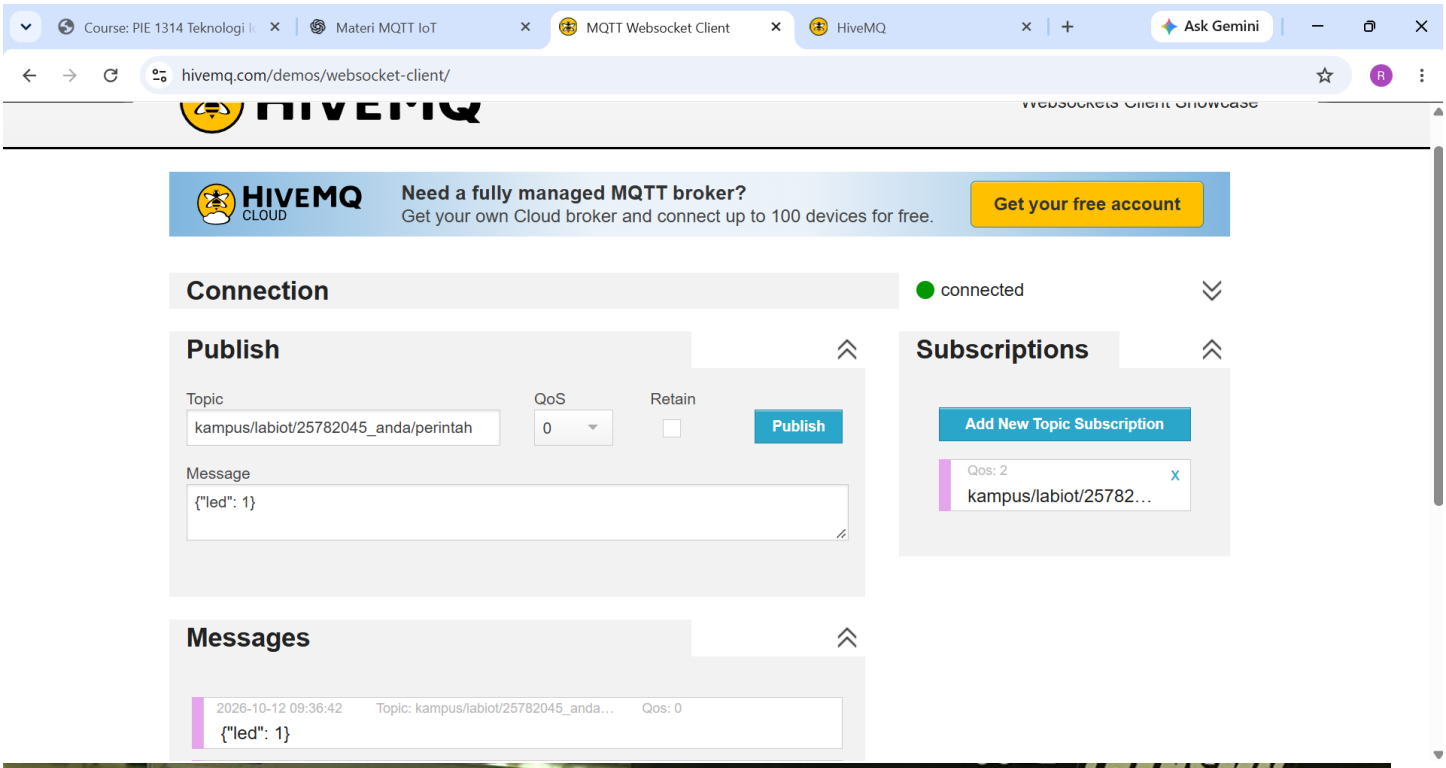
  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageReceived); // Daftarkan fungsi callback
}

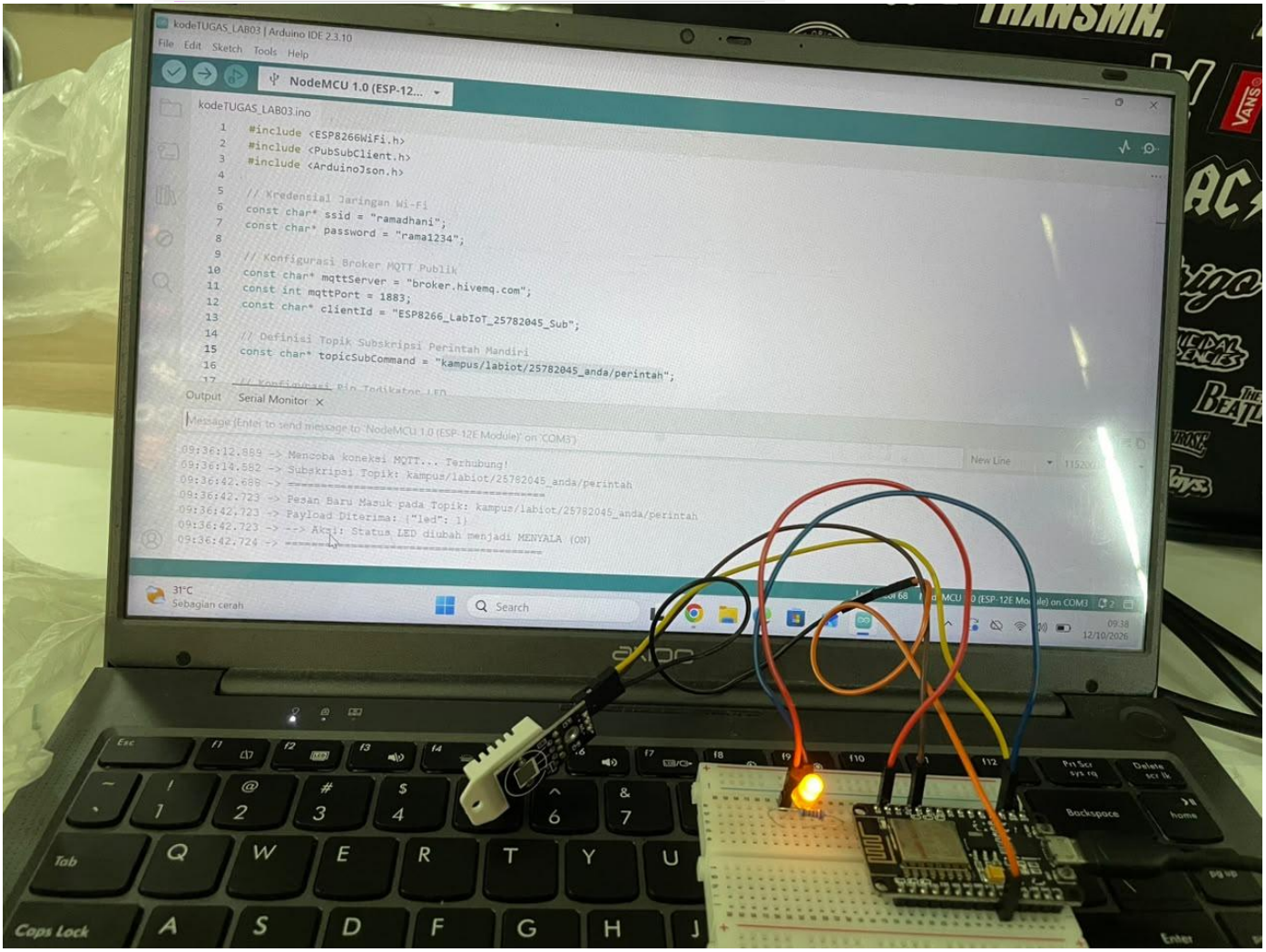
void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Wajib untuk mendengarkan pesan masuk
}
```

2. Unggah kode ke board NodeMCU Anda.

Penjelasan Singkat Kode:

- client.setCallback(onMessageReceived); : Mendaftarkan fungsi penangan paket data masuk (*callback function*). Kapan pun broker mengirimkan pesan yang sesuai dengan topik subskripsi, fungsi onMessageReceived akan dipicu secara otomatis.
- client.subscribe(topicSubCommand); : Mengirimkan instruksi subskripsi ke broker bahwa klien bersedia menerima semua pesan yang dialirkan ke topik tersebut.
- deserializeJson(doc, payloadStr); : Mengurai teks string JSON mentah dan menyusunnya kembali menjadi struktur pohon data yang kuncinya dapat diperiksa melalui doc.containsKey() dan nilainya dapat diekstrak secara langsung.


Course: PIE 1314 Teknologi I
Materi MQTT IoT
MQTT Websocket Client
HiveMQ
Ask Gemini
hivemq.com/demos/websocket-client/
HIVEMQ CLOUD
Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.
Get your free account
Connection
connected
Publish
Topic
kampus/labiot/25782045_anda/perintah
QoS
0
Retain
Publish
Message
{"led": 1}
Subscriptions
Add New Topic Subscription
Qos: 2
kampus/labiot/25782…
Messages
2026-10-12 09:36:42
Topic: kampus/labiot/25782045_anda…
Qos: 0
{"led": 1}


kodeTUGAS_LAB03 | Arduino IDE 2.3.10
NodeMCU 1.0 (ESP-12...
kodeTUGAS_LAB03.ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
// Kredensial Jaringan Wi-Fi
const char* ssid = "ramadhani";
const char* password = "rama1234";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782045_Sub";
// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782045_anda/perintah";
Output
Serial Monitor
09:36:12.889 -> Mencoba koneksi MQTT... Terhubung!
09:36:14.582 -> Subskripsi Topik: kampus/labiot/25782045_anda/perintah
09:36:42.723 -> Pesan Baru Masuk pada Topik: kampus/labiot/25782045_anda/perintah
09:36:42.723 -> Payload Diterima: {"led": 1}
09:36:42.723 -> --> Aksi: Status LED diubah menjadi MENYALA (ON)
31°C
Sebagian cerah
Search

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

Langkah Kerja Pemrograman:

1. Masukkan kode terpadu berikut ke Arduino IDE:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_NIM01_Full";

// Pembagian Topik Mandiri
const char* topicPublish   = "kampus/labiot/nim_anda/telemetri";
const char* topicSubscribe = "kampus/labiot/nim_anda/kendali";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik

void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);
```

```
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  if (doc.containsKey("led")) {
    int val = doc["led"];
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
    Serial.printf("--> [INDIKATOR] Status LED diubah: %s\n", val == 1 ? "MENYALA" : "PADAM");
  }
  Serial.println("----------------------------------------");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik: %s\n", topicSubscribe);
    } else {
      Serial.printf("Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address: ");
```

```
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop(); // Memelihara sesi TCP dan memproses callback masuk

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

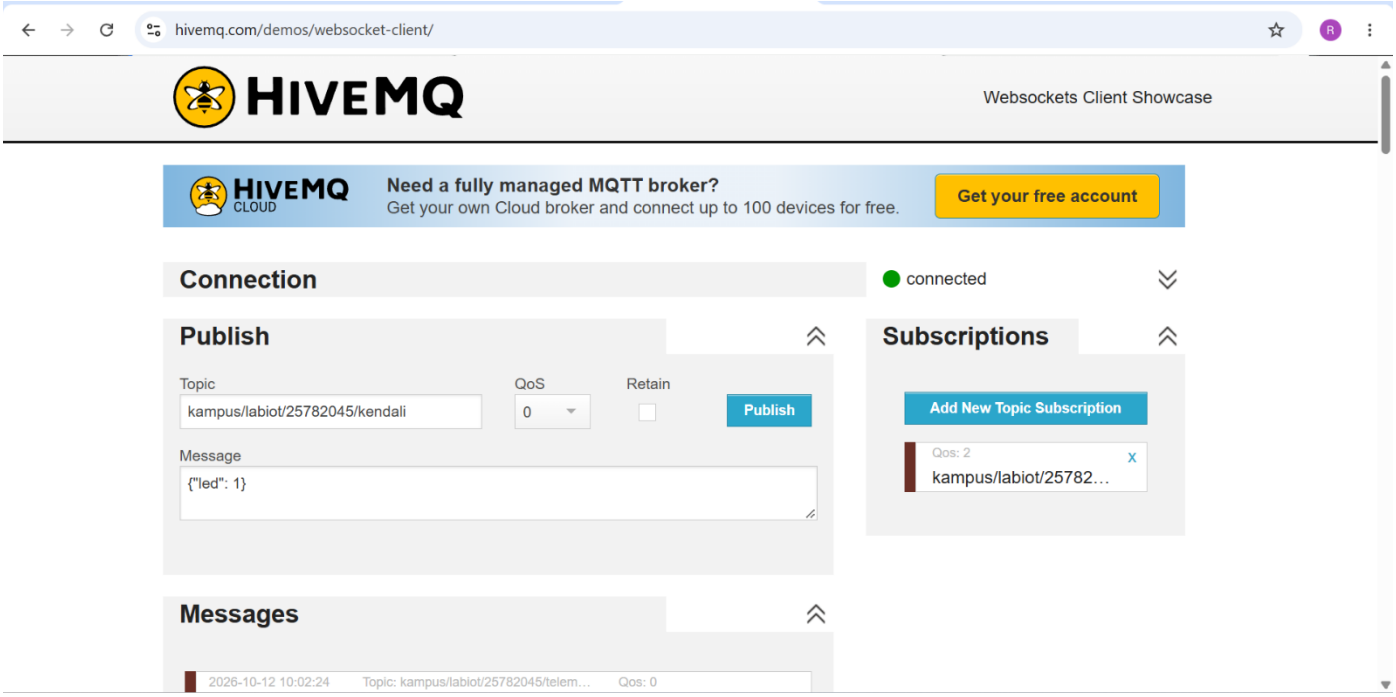
    float t = dht.readTemperature();
    float h = dht.readHumidity();

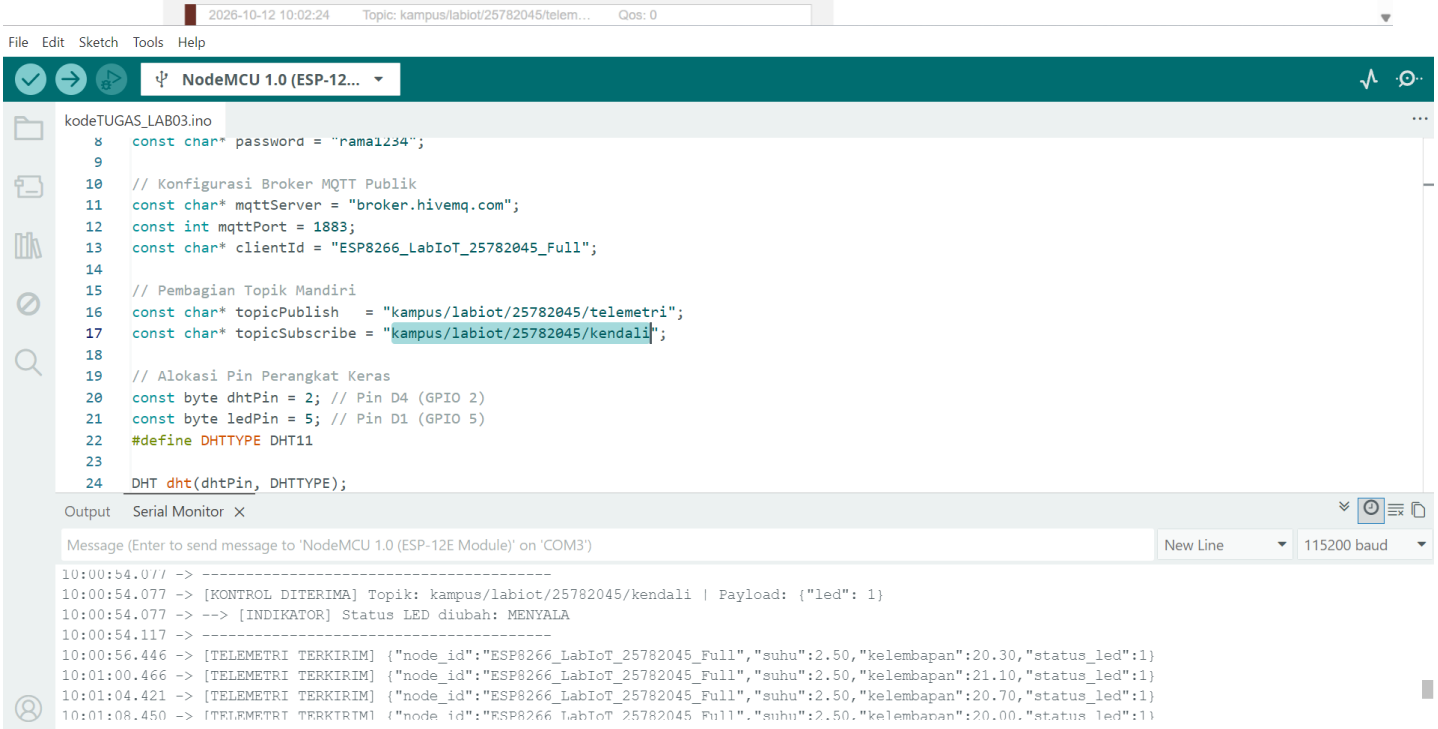
    if (!isnan(t) && !isnan(h)) {
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);

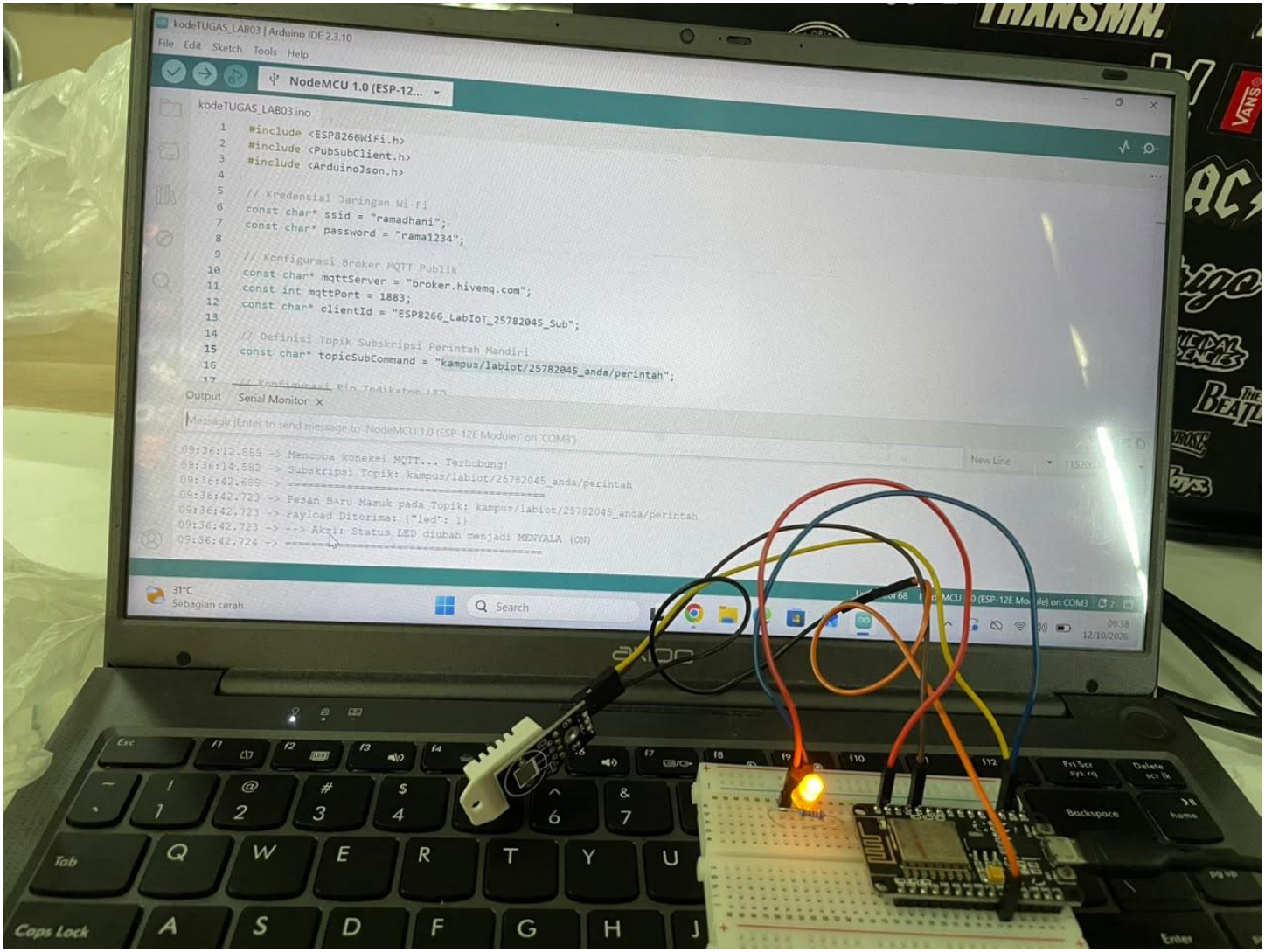
      char buffer[256];
      serializeJson(docOut, buffer);

      bool success = client.publish(topicPublish, buffer);
      if (success) {
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri!");
      }
    }
  }
}
```

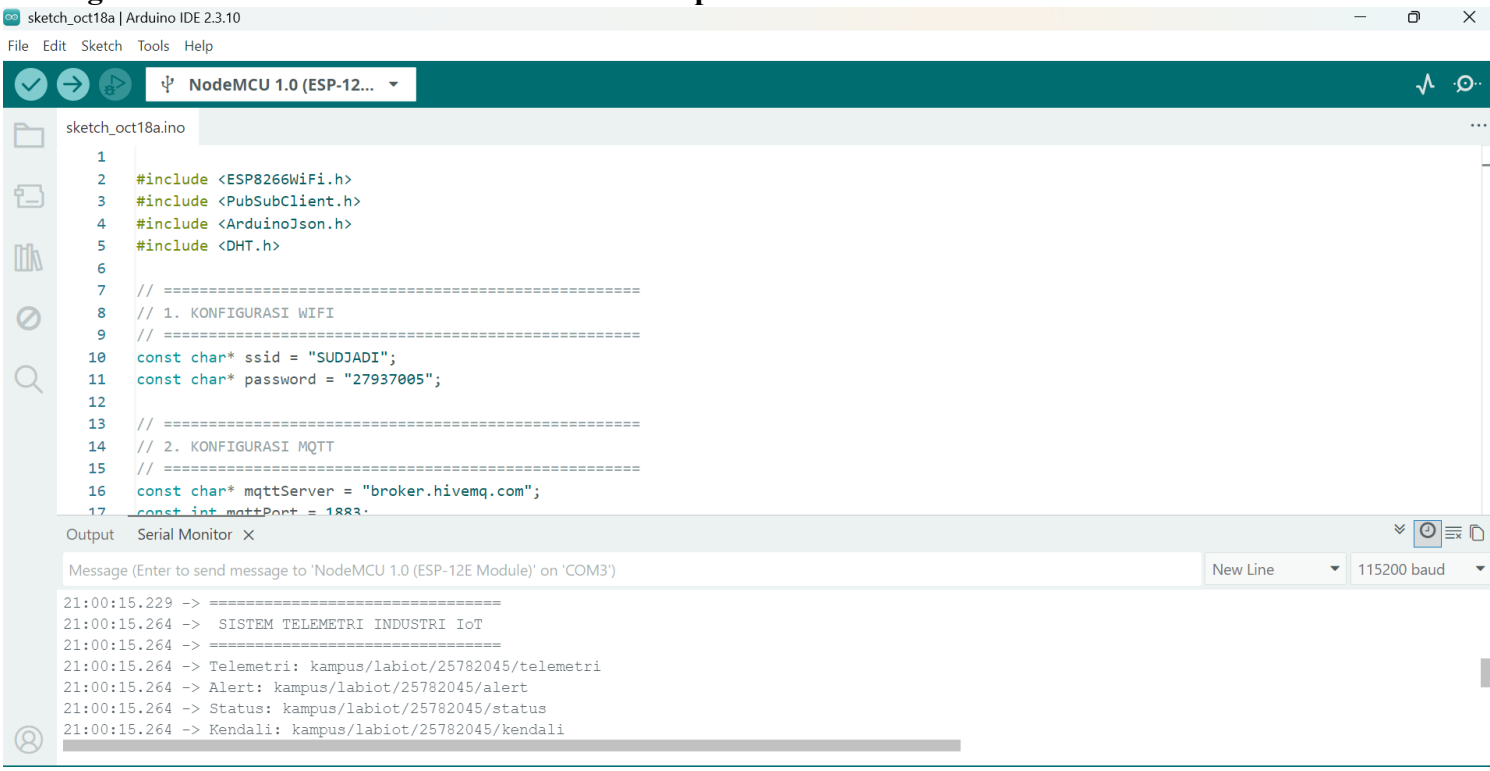
Hasil dari praktikum

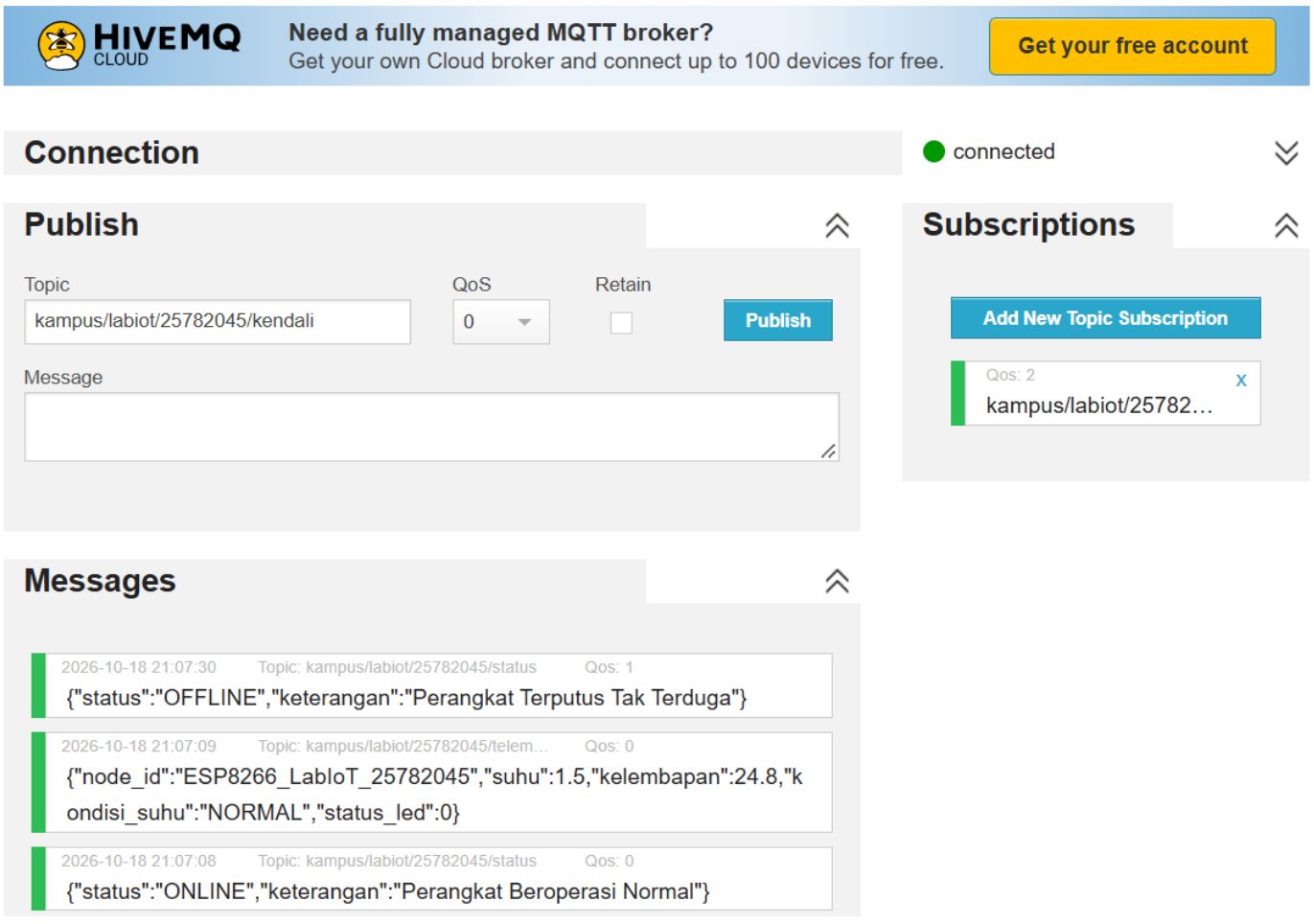





kodeTUGAS_LAB03 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
kodeTUGAS_LAB03.ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
// Kredensial Jaringan Wi-Fi
const char* ssid = "ramadhani";
const char* password = "rama1234";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782045_Sub";
// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782045_anda/perintah";
Output Serial Monitor
09:36:12.889 -> Mencoba koneksi MQTT... Terhubung!
09:36:14.582 -> Subskripsi Topik: kampus/labiot/25782045_anda/perintah
09:36:42.689 -> ==========
09:36:42.723 -> Pesan Baru Masuk pada Topik: kampus/labiot/25782045_anda/perintah
09:36:42.723 -> Payload Diterima: {"led": 1}
09:36:42.723 -> --> Aksi: Status LED diubah menjadi MENYALA (ON)
09:36:42.724 -> ==========
31°C
Sebagian cerah
Search
12/10/2026

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri





Penjelasan
pengembangan tingkat lanjut dari Praktikum 3 yang bertujuan untuk mengubah sistem IoT menjadi sarana pengawasan keselamatan industri yang cerdas, otonom, dan tahan banting. Pengembangan utama dari praktikum sebelumnya mencakup penambahan klasifikasi status lingkungan secara dinamis pada data telemetri rutin ("NORMAL" atau "PANAS" berdasarkan ambang batas suhu 31.0°C), serta penerapan sistem kecerdasan lokal (edge safety engine) yang membuat mikrokontroler dapat secara otonom mempublikasikan peringatan bahaya ke topik khusus (alert) dan menyalakan LED darurat tanpa menunggu instruksi operator ketika terjadi overheat. Selain itu, sistem ini dilengkapi dengan mekanisme ketahanan jaringan Last Will and Testament (LWT) agar broker MQTT dapat secara otomatis menyiarkan status OFFLINE jika perangkat terputus secara mendadak, sementara fleksibilitas kendali jarak jauh manual (override) tetap dipertahankan bagi operator melalui topik kendali selama kondisi suhu berada di batas aman.